

PLAYWRIGHT + PYTHON

TABLEAU ONLINE

Cheatsheet de supervivencia para QA automation

Async API · Locators · ARIA · Dropdowns · Multiselect · Tableau · Debugging · Best practices

Objetivo: Que puedas resolver una bronca de Playwright en el trabajo sin depender de documentación externa. Está pensado especialmente para Tableau Online, donde el DOM cambia, hay componentes custom y aparecen overlays durante los rerenders.

1. Modelo mental: Locator → Action → Assertion

```
checkbox = page.get_by_role("checkbox", name="Muscular") # LOCATOR
await checkbox.click() # ACTION
await expect(checkbox).to_have_attribute("aria-checked", "false") # ASSERTION
```

- Locator: describe qué elemento quieres. No “agarra” necesariamente el nodo en ese instante; se reevalúa cuando lo usas.
- Action: click, fill, check, press, hover, etc.
- Assertion: expect(...). Sirve para validar que la UI terminó en el estado esperado.

2. Base recomendada para Spyder / IPython

```
from playwright.async_api import async_playwright, expect

URL = "https://public.tableau.com/..."

async def main():
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=False, slow_mo=250)
        context = await browser.new_context(viewport={"width": 1800, "height": 1200})
        page = await context.new_page()
        await page.goto(URL)

        # ... automation ...

        input("Enter para cerrar")
        await browser.close()

# En Spyder/IPython si ya hay event loop:
await main()

# En un .py ejecutado desde Terminal:
# import asyncio
# asyncio.run(main())
```

Ojo con Spyder: Si aparece “asyncio.run() cannot be called from a running event loop”, usa await main() en esa consola. En un script normal ejecutado desde Terminal, asyncio.run(main()) sigue siendo el patrón

estándar.

3. Jerarquía de locators: cuál usar primero

Prioridad	Locator	Cuándo usarlo
1	get_by_role(...)	Botones, checkboxes, comboboxes, dialogs, links. Excelente si ARIA está bien.
2	get_by_label(...)	Inputs/form controls con etiqueta accesible.
3	get_by_text(..., exact=True)	Texto visible estable.
4	get_by_placeholder / get_by_title	Cuando esos atributos son estables y específicos.
5	CSS por clase/atributo estable	Tableau: .FIIItem, [role="checkbox"], button.apply, etc.
6	filter(has_text=...)	Para distinguir elementos con misma estructura.
7	XPath	Solo cuando no hay una alternativa limpia.
8	Coordenadas / force=True	Último recurso; muy frágil.

4. ARIA en Tableau: tu mejor pista

```
<span role="combobox" aria-expanded="false" aria-controls="..._menu">  
<div role="checkbox" aria-checked="true"> ... </div>
```

- role="combobox" → el control se comporta como dropdown.
- aria-expanded="false/true" → cerrado / abierto.
- aria-controls="ID" → suele apuntar al ID del menú asociado. Esto es oro para evitar IDs hardcodeados.
- role="checkbox" + aria-checked="true/false" → checkbox custom de Tableau.
- aria-multiselectable="true" → listbox permite múltiples selecciones.

5. Dropdown: comprobar que se desplegó

```
dropdown = page.get_by_role("combobox").filter(has_text="Multiple values")

await dropdown.click()

# 1) Confirmar el estado del control
await expect(dropdown).to_have_attribute("aria-expanded", "true")

# 2) Encontrar el menú vinculado dinámicamente
menu_id = await dropdown.get_attribute("aria-controls")
menu = page.locator(f"#{menu_id}")

# 3) Confirmar que el menú ya está visible
await expect(menu).to_be_visible()
```

Best practice Tableau: No hardcodees `#tableau_base_widget_LegacyCategoricalQuickFilter_0_menu` si puedes obtenerlo desde `aria-controls`. Tableau puede regenerar IDs entre sesiones o dashboards.

6. “Guardar” los elementos cuando el dropdown abre

- Si solo necesitas seguir interactuando: guarda Locators, no ElementHandles. Los Locators se reevalúan y toleran mejor rerenders.
- Si necesitas una fotografía del estado actual: guarda texto + atributos en listas/diccionarios.

```
checkboxes = menu.locator('div[role="checkbox"]')

# Locator collection (sigue siendo dinámico)
print("Cantidad:", await checkboxes.count())
print("Textos:", await checkboxes.all_inner_texts())
```

6.1 Snapshot simple: lista de textos

```
options = [
    text.strip()
    for text in await checkboxes.all_inner_texts()
]

print(options)
# ['(All)', 'Circulatory', 'Digestive', ...]
```

6.2 Snapshot completo: nombre + seleccionado

```
options = []

for i in range(await checkboxes.count()):
    item = checkboxes.nth(i)

    options.append({
        "name": (await item.inner_text()).strip(),
        "checked": (await item.get_attribute("aria-checked")) == "true"
    })

print(options)

[
    {"name": "(All)", "checked": True},
    {"name": "Circulatory", "checked": True},
    {"name": "Muscular", "checked": False}
]
```

6.3 Guardarlo en JSON para evidencia / debugging

```
import json

with open("dropdown_snapshot.json", "w", encoding="utf-8") as f:
    json.dump(options, f, ensure_ascii=False, indent=2)
```

6.4 Regla importante: NO guardes nodos viejos si Tableau rerenderiza

```
# EVITA guardar ElementHandles para reutilizarlos después de Apply/rerender.
# Mejor guarda el valor lógico:
target_name = "Muscular"

# Y vuelve a construir el locator cuando lo necesites:
item = menu.locator('div[role="checkbox"]', has_text=target_name)
```

Por qué: Tableau puede destruir y volver a crear el DOM después de un filtro. Un locator describe cómo reencontrar el elemento; un handle puede quedar “detached”.

7. Clickear un checkbox específico dentro del multiselect

```
checkboxes = menu.locator('div[role="checkbox"]')

muscular = checkboxes.filter(has_text="Muscular")

await expect(muscular).to_be_visible()
await muscular.click()

await expect(muscular).to_have_attribute(
    "aria-checked",
    "false"
)
```

- En Tableau prefiero clickear el contenedor con role="checkbox" antes que el <input> interno, porque Tableau maneja eventos/clases propias.
- Para checkbox HTML nativo (<input type="checkbox">), sí usa check()/uncheck() cuando corresponda.

8. Seleccionar N valores de forma reusable

```
async def click_values(menu, values):
    for value in values:
        item = menu.locator('div[role="checkbox"]', has_text=value)
        await expect(item).to_be_visible()
        await item.click()

await click_values(menu, ["Muscular", "Skeletal", "Nervous"])
await menu.get_by_role("button", name="Apply").click()
```

9. Identificar varios dropdowns en la misma página

- No uses page.get_by_role("combobox").first salvo que estés 100% seguro de que el orden nunca cambia.
- Prefiere scope: encuentra primero el contenedor/panel del filtro y después el combobox dentro.

```
filter_panel = page.locator(".some-filter-container").filter(has_text="Anatomy System")
dropdown = filter_panel.get_by_role("combobox")
```

- Si existe un nombre accesible real, usa get_by_role("combobox", name="...").

- Si Tableau genera aria-labelledby, puedes inspeccionar a qué texto apunta y usar ese contexto para localizar el control.

10. Apply: validar que Tableau terminó

```
apply_btn = menu.get_by_role("button", name="Apply")
await apply_btn.click()

# Confirmar que el menú cerró
await expect(menu).to_be_hidden()

# Si el combobox conserva el mismo locator:
await expect(dropdown).to_have_attribute("aria-expanded", "false")
```

- Después de Apply, vuelve a localizar elementos que dependan de la visualización filtrada.
- Si aparece una capa tipo tabGlass/overlay, espera a que desaparezca antes de la siguiente interacción.

11. No-no's: cosas que conviene evitar

- Hardcodear IDs dinámicos de Tableau (tab-ui-id-..., LegacyCategoricalQuickFilter_0_menu) cuando existe un atributo semántico o aria-controls.
- Usar nth(0), first o last como solución automática a un locator ambiguo. Si el orden cambia, el test hace click en otra cosa.
- Usar force=True para “arreglar” todo. Puede ocultar overlays, bugs de sincronización o un locator incorrecto.
- Usar coordenadas mouse.click(x, y) para controles normales. Se rompen con viewport, zoom o cambios de layout.
- Poner time.sleep(5) por todos lados. Espera estados reales con expect(...) o wait_for(...).
- Guardar ElementHandles y reutilizarlos después de un filtro/Apply de Tableau. El DOM puede haber sido reemplazado.
- Buscar en toda la página cuando puedes hacer scope dentro del menú/panel correcto.
- Clickear el <input> interno de un componente custom sin comprobar que Tableau realmente escucha ese evento.
- Confiar solo en “el click no dio error”. Siempre valida el estado esperado.
- Usar networkidle como señal universal de “Tableau terminó”; dashboards pueden mantener requests activos.
- Ignorar iframes. Si el dashboard está embebido, el elemento puede existir pero estar dentro de otro frame.
- Automatizar con texto parcial demasiado genérico: has_text="All" puede coincidir con más cosas de las que esperas.

12. Best practices para Tableau Online

- Prefiere roles/ARIA y texto de negocio sobre IDs generados.
- Obtén el menú asociado desde aria-controls cuando sea posible.
- Haz scope: dashboard → filtro → dropdown → menú → opción.
- Después de cada Apply, asume que Tableau puede rerenderizar y reencuentra lo que vas a usar.
- Valida dos cosas cuando puedas: estado del control (aria-checked/aria-expanded) y resultado visible del dashboard.
- Usa exact=True o locators más específicos cuando haya textos repetidos.
- Antes de clickear, deja que Playwright haga auto-wait; agrega expect(...).to_be_visible() cuando el estado sea relevante para el test.

- Captura screenshots/tracing cuando una secuencia falle de forma intermitente.
- Construye helpers para filtros repetidos; evita copiar 40 bloques casi iguales.
- Guarda snapshots lógicos (nombre, selected, count) si necesitas evidencia o comparar antes/después.

13. Qué identificador usar según el caso

Caso	Preferido	Ejemplo
Botón Apply	role + name	menu.get_by_role("button", name="Apply")
Checkbox custom Tableau	role + texto	menu.locator('div[role="checkbox"]', has_text="Muscular")
Dropdown	role="combobox" + scope/name	panel.get_by_role("combobox")
Menú abierto	aria-controls del dropdown	page.locator(f'#{menu_id}')
Input Search	role/aria-label	menu.get_by_role("combobox", name="Search (Enter)") o locator por aria-label
Texto visible único	get_by_text exact	page.get_by_text("Skeletal", exact=True)
Clase Tableau estable	CSS	menu.locator(".FIItem")
ID tab-ui-id-*	Evitar	Solo para debugging puntual, no como selector principal

14. Debugging cuando “no hace click”

```
print("count:", await locator.count())
print("visible:", await locator.is_visible())
print("enabled:", await locator.is_enabled())
print("text:", await locator.inner_text())
print("html:", await locator.evaluate("el => el.outerHTML"))
await locator.highlight()
```

- count == 0 → selector incorrecto, frame incorrecto o todavía no apareció.
- count > 1 → strict mode / locator ambiguo; agrega scope, name, exact o filter.
- Existe pero invisible → menú cerrado, template oculto o duplicado invisible.
- Visible pero click interceptado → overlay, tabGlass, spinner, modal o animación.
- Click cambia y vuelve → rerender, dependencia entre filtros, falta Apply o estado no confirmado.

15. Patrón de diagnóstico de overlays / tabGlass

```
glass = page.locator(".tabGlass")

if await glass.count() > 0:
    # Solo si sabes que la capa debe desaparecer al terminar el render
    await expect(glass).to_be_hidden(timeout=15000)

await target.click()
```

No conviertas .tabGlass en dogma: La clase exacta puede variar. Primero inspecciona el overlay real; la idea

es esperar la condición de “interacción disponible”, no memorizar una sola clase.

16. Iframes: el clásico “lo veo pero Playwright no”

```
for frame in page.frames:
    print(frame.url)

frame = page.frame_locator("iframe")
await frame.get_by_role("button", name="Apply").click()
```

- Tableau Public/Online puede aparecer directamente o embebido. Si está embebido en otra app, revisa frames temprano.

17. Helpers recomendados

```
async def open_tableau_dropdown(dropdown):
    await dropdown.click()
    await expect(dropdown).to_have_attribute("aria-expanded", "true")
    menu_id = await dropdown.get_attribute("aria-controls")
    menu = dropdown.page.locator(f"#{menu_id}")
    await expect(menu).to_be_visible()
    return menu

async def snapshot_multiselect(menu):
    items = menu.locator('div[role="checkbox"]')
    result = []
    for i in range(await items.count()):
        item = items.nth(i)
        result.append({
            "name": (await item.inner_text()).strip(),
            "checked": (await item.get_attribute("aria-checked")) == "true"
        })
    return result
```

Nota de implementación: Dependiendo de tu versión/estilo de código, puede ser más simple pasar también page al helper en vez de intentar derivarlo desde dropdown. La idea importante es separar “abrir”, “obtener menú” y “snapshot”.

18. Versión robusta y explícita del helper

```
async def open_tableau_dropdown(page, dropdown):
    await dropdown.click()
    await expect(dropdown).to_have_attribute("aria-expanded", "true")

    menu_id = await dropdown.get_attribute("aria-controls")
    if not menu_id:
        raise RuntimeError("El combobox no tiene aria-controls")

    menu = page.locator(f"#{menu_id}")
    await expect(menu).to_be_visible()
    return menu
```

19. Flujo completo de ejemplo

```
# 1. Localizar dropdown dentro de su panel
panel = page.locator(".CFContent").filter(has_text="Multiple values")
dropdown = panel.get_by_role("combobox")

# 2. Abrir y obtener menú asociado
menu = await open_tableau_dropdown(page, dropdown)

# 3. Guardar estado inicial
before = await snapshot_multiselect(menu)
print("ANTES:", before)

# 4. Clickear opción específica
muscular = menu.locator('div[role="checkbox"]', has_text="Muscular")
await muscular.click()
await expect(muscular).to_have_attribute("aria-checked", "false")

# 5. Guardar estado previo al Apply
after_click = await snapshot_multiselect(menu)
print("DESPUÉS DEL CLICK:", after_click)

# 6. Apply
await menu.get_by_role("button", name="Apply").click()
await expect(menu).to_be_hidden()

# 7. Después del rerender, vuelve a localizar lo que necesites
#     en vez de confiar en nodos viejos.
```

20. Mini referencia rápida

Necesito...	Haz...
Abrir dropdown	<code>await dropdown.click()</code>
Comprobar abierto	<code>expect(dropdown).to_have_attribute("aria-expanded", "true")</code>
Saber qué menú controla	<code>await dropdown.get_attribute("aria-controls")</code>
Esperar menú	<code>expect(menu).to_be_visible()</code>
Listar opciones	<code>await menu.locator('div[role="checkbox"]').all_inner_texts()</code>
Saber cuántas	<code>await locator.count()</code>
Elegir una por texto	<code>locator.filter(has_text="Muscular")</code>
Leer selección	<code>get_attribute("aria-checked")</code>
Aplicar	<code>menu.get_by_role("button", name="Apply").click()</code>
Confirmar cierre	<code>expect(menu).to_be_hidden()</code>
Debug visual	<code>await locator.highlight()</code>
Pausar e inspeccionar	<code>await page.pause()</code>
Screenshot	<code>await page.screenshot(path="debug.png", full_page=True)</code>
Evitar sleep	espera estado/atributo/visibilidad real

Regla de oro para Tableau: localiza por significado, espera por estado y asume rerender después de filtros.